Finance ERP Backend Architecture

Node.js, Express, MySQL, bcrypt and secure cookie sessions

> **System purpose:** this is a contribution record-management system. It records contributions that were already received through cash, M-Pesa, bank transfer, cheque or another external method. It does not initiate or process payments.

1. Architecture overview

```
React Frontend
      |
      | HTTPS + secure session cookie
      v
Express API
      |
      +-- Session middleware
      +-- Authentication and authorization
      +-- Request validation
      +-- Application services
      +-- Reports and dashboards
      +-- Error handling
      |
      v
MySQL 8+
```

Recommended project structure:

```
Backend/
├── src/
│   ├── app.js
│   ├── server.js
│   ├── config/
│   ├── db/
│   ├── middleware/
│   ├── routes/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
│   ├── validators/
│   ├── serializers/
│   ├── errors/
│   └── utils/
├── migrations/
├── seeders/
├── tests/
├── .env.example
└── README.md
```

Routes define HTTP endpoints. Middleware handles sessions, authentication, roles, scopes and validation. Controllers translate requests into service calls. Services contain business rules. Repositories contain MySQL access. Serializers prevent sensitive fields from being returned.

2. Database strategy

Use one MySQL database with shared normalized tables:

```
parishes
users
centres
groups
members
```

```
contributions
audit_logs
```

Every parish-owned table includes `parish_id`. Creating a parish creates a tenant record in `parishes`; it does not create a physical database table. Use InnoDB for all tables.

3. Tenant and hierarchy model

Role	Scope
owner	Entire platform
parish	One parish
centre	One parish and one centre
group	One parish, one centre and one group

```
parishes
    │
    ├── users
    ├── centres
    │      └── groups
    │              └── members
    │                      └── contributions
    └── audit_logs
```

Relationships: centres.parish_id -> parishes.id; groups.parish_id -> parishes.id; groups.centre_id -> centres.id; members.parish_id/centre_id/group_id -> their parents; contributions.parish_id/centre_id/group_id/member_id -> their parents; contributions.created_by -> users.id.

The backend must reject inconsistent combinations, such as a member from one parish being placed in a group from another parish.

4. Database tables

parishes

```
id, name, code, location, priest, status, created_at, updated_at
```

Use a UUID-like `CHAR(36)` primary key, unique code, and statuses: active, inactive, suspended.

users

```
id, parish_id, centre_id, group_id, name, email, password_hash,
role, status, last_login_at, created_at, updated_at
```

Owner users have no parish, centre or group. Parish users have parish_id. Centre users have parish_id and centre_id. Group users have all three. Use a unique normalized email index. Store only bcrypt hashes.

centres

```
id, parish_id, name, location, status, created_at, updated_at
```

Use `UNIQUE(parish_id, name)`. Centre login accounts belong in `users`, not this table.

groups

```
id, parish_id, centre_id, name, status, created_at, updated_at
```

Use `UNIQUE(centre_id, name)`. A group must belong to the same parish as its centre.

members

```
id, parish_id, centre_id, group_id, name, phone, status,
created_at, updated_at
```

Add indexes on parish_id, parish_id + centre_id, and parish_id + centre_id + group_id.

contributions

```
id, parish_id, centre_id, group_id, member_id, amount, currency,
category, contribution_date, method, reference, notes, status,
created_by, recorded_at, voided_by, voided_at, void_reason,
created_at, updated_at
```

Use `DECIMAL(14,2)` for amount, not floating point. Default currency is KES. Methods can include cash, m_pesa, bank_transfer, cheque and other. The reference is an external receipt, deposit slip, M-Pesa or cheque reference.

audit_logs

```
id, user_id, parish_id, action, entity_type, entity_id, metadata, created_at
```

Record login, logout, parish provisioning, record creation, updates, voids and other important changes. For contribution changes, metadata should contain old and new values.

5. Session architecture

Use `express-session` with a MySQL-backed session store such as `express-mysql-session`. Do not use Express's default in-memory store in production.

```
Cookie:
  httpOnly: true
  secure: true in production
  sameSite: 'lax'
  maxAge: configured session duration

Session data:
  { "userId": "user-uuid" }
```

The cookie contains only a session identifier. Each request loads the session and active user from MySQL and builds an authorization context containing userId, role, parishId, centreId and groupId.

6. Password security with bcrypt

1. Receive the password over HTTPS.
2. Validate password requirements.
3. Hash it with bcrypt.
4. Store only password_hash.
5. Use bcrypt.compare during login.

Use a configurable cost such as `BCRYPT_ROUNDS=12`. Never store, return, log or audit plain-text passwords or hashes.

7. Authorization

Role	Allowed operations
Owner	Manage parishes and platform summaries; create, update, suspend or archive parish tenants.
Parish administrator	Manage centres and parish-scoped contribution records.
Centre administrator	Manage groups and centre-scoped contribution records.
Group treasurer	Manage group members and group contribution records.

Authorization is always enforced by the backend. Frontend route guards are only for user experience.

8. Authentication API

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Verify email/password with bcrypt and create a MySQL session.
GET	/api/v1/auth/me	Return the safe current user profile.
POST	/api/v1/auth/logout	Destroy the session and clear the cookie.
PATCH	/api/v1/auth/password	Verify current password and save a new bcrypt hash.

Login returns a safe user object but no password, password_hash or access token.

9. Parish creation workflow

Only the ERP owner can call `POST /api/v1/parishes`.

```
{
  "name": "St. Mary Parish",
  "code": "STM-001",
  "location": "Nairobi",
  "priest": "Fr. Peter Mwangi",
  "administrator": {
    "name": "Agnes Njeri",
    "email": "admin.stmary@parishledger.org",
    "password": "secure-password"
  }
}
```

1. Authenticate requester and confirm owner role.
2. Validate parish and administrator data.
3. Check code and email uniqueness.
4. Begin a database transaction.
5. Insert parish.
6. Hash password with bcrypt and insert administrator user.
7. Insert audit log.
8. Commit and return safe data.

If an operation fails, roll back the entire transaction. No physical table is created; the new tenant is represented by `parishes.id`.

10. Resource APIs

Parishes

```
GET    /api/v1/parishes
POST   /api/v1/parishes
GET    /api/v1/parishes/:parishId
PATCH  /api/v1/parishes/:parishId
DELETE /api/v1/parishes/:parishId
PATCH  /api/v1/parishes/:parishId/status
```

Centres

```
GET    /api/v1/centres
POST   /api/v1/centres
GET    /api/v1/centres/:centreId
PATCH  /api/v1/centres/:centreId
DELETE /api/v1/centres/:centreId
PATCH  /api/v1/centres/:centreId/status
```

Groups

```
GET    /api/v1/groups
POST   /api/v1/groups
GET    /api/v1/groups/:groupId
PATCH  /api/v1/groups/:groupId
DELETE /api/v1/groups/:groupId
PATCH  /api/v1/groups/:groupId/status
```

Members

```
GET    /api/v1/members
POST   /api/v1/members
GET    /api/v1/members/:memberId
PATCH  /api/v1/members/:memberId
DELETE /api/v1/members/:memberId
PATCH  /api/v1/members/:memberId/status
```

For centre, group and member creation, derive hierarchy IDs from the authenticated scope where possible rather than trusting browser-supplied IDs.

11. Contribution record API

```
GET   /api/v1/contributions
POST  /api/v1/contributions
GET   /api/v1/contributions/:contributionId
PATCH /api/v1/contributions/:contributionId
POST  /api/v1/contributions/:contributionId/void
```

Supported list filters: page, pageSize, q, category, method, parishId, centreId, groupId, memberId, fromDate, toDate and sort. Scope restrictions are applied before client filters.

```
{
  "memberId": "member-id",
  "amount": 12500,
  "category": "tithes",
  "date": "2026-10-24",
  "method": "m_pesa",
  "reference": "QWE123",
  "notes": "October Sunday contribution"
}
```

When recording a contribution, the backend loads the member, confirms scope, derives parish/centre/group IDs, inserts the historical record, sets created_by and recorded_at, writes an audit log and returns the saved record. Do not trust hierarchy IDs from the frontend.

Do not permanently delete contributions. Use the void endpoint with a required reason. Voided records remain auditable but are excluded from normal totals.

12. Dashboard and overview APIs

```
GET /api/v1/dashboard/owner
GET /api/v1/dashboard/parish
GET /api/v1/dashboard/centre
GET /api/v1/dashboard/group
```

Return summary counts, recorded contribution totals, monthly trends, recent records and child summaries. Exclude status = voided from normal calculations.

```
GET /api/v1/parishes/:parishId/overview
GET /api/v1/centres/:centreId/overview
GET /api/v1/groups/:groupId/overview
GET /api/v1/members/:memberId/overview
```

13. Reports

```
GET /api/v1/reports/contributions.csv
GET /api/v1/reports/contributions.summary
GET /api/v1/reports/parishes/:parishId/summary
GET /api/v1/reports/centres/:centreId/summary
GET /api/v1/reports/groups/:groupId/summary
```

Reports use the same authorization and filters as contribution lists. No payment-success, settlement, refund, gateway-fee or payment-failure reports are needed.

14. Request workflow

```
HTTP request
  -> session middleware
  -> load user and status
  -> authentication
  -> role authorization
  -> scope authorization
  -> request validation
  -> controller
  -> service
  -> repository
  -> MySQL
  -> audit log
  -> serializer
  -> response
```

Hierarchy validation must occur before every create or update. Use transactions where multiple writes must succeed together.

15. Validation and errors

Validate required names, codes, emails, statuses, parent relationships, positive decimal amounts, dates, methods and references using Zod or Joi. Use one error format:

```
{
  "error": {
    "code": "FORBIDDEN",
    "message": "You do not have permission to update this member.",
    "details": []
  }
}
```

Status codes: 400 malformed request; 401 unauthenticated; 403 unauthorized; 404 not found; 409 conflict; 422 validation failure; 500 unexpected error.

16. Deletion and audit policy

Prefer inactive or suspended status for parishes, centres, groups and members. Parish deletion must be owner-only, explicitly confirmed, audited and controlled. Contributions should be voided rather than deleted, preserving financial accountability and history.

17. Security configuration

- HTTPS in production
- HttpOnly, Secure and SameSite cookies
- Session expiration and MySQL session store
- bcrypt password hashing
- Login rate limiting
- Parameterized SQL
- Restricted CORS
- CSRF protection for cookie-authenticated state-changing requests
- Helmet security headers
- Request size limits and audit logging

Allow only the real frontend origin and never use wildcard CORS with credentials. Store secrets in environment variables.

18. Environment configuration

```
NODE_ENV=development
PORT=4000
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_DATABASE=finance_erp
MYSQL_USER=finance_app
MYSQL_PASSWORD=...
SESSION_SECRET=...
SESSION_COOKIE_NAME=finance_erp_session
SESSION_MAX_AGE_MS=86400000
BCRYPT_ROUNDS=12
FRONTEND_ORIGIN=http://localhost:5173
```

Never commit .env, database passwords, session secrets or production credentials. Commit only .env.example.

19. Frontend integration

1. Create a central API client with base URL, `credentials: include`, JSON, errors, 401 handling and CSRF support.
2. Replace local authentication with login, me and logout API calls.
3. Update `useRecords` to use HTTP instead of localStorage.
4. Replace local dashboard calculations with dashboard endpoints.

5. Keep local CSV/print exports initially; move reports server-side later.
6. Add loading, empty, validation, permission, server-error and session-expiration states.

20. Implementation phases

1. **Foundation:** initialize Node.js, install Express/MySQL packages, configure environment, add health endpoint and error handling.
2. **Database:** create migrations, tables, indexes, constraints and seed data.
3. **Authentication:** MySQL sessions, bcrypt, login, logout and current-user endpoint.
4. **Authorization:** authentication, role, parish, centre and group middleware.
5. **Hierarchy APIs:** parish, centre, group and member services.
6. **Contribution records:** record, list, filter, update, void and audit.
7. **Dashboards and reports:** aggregates, CSV and summaries.
8. **Frontend integration:** API client, loading/error states and session handling.
9. **Testing/deployment:** unit, authorization, hierarchy, contribution, session and integration tests; HTTPS, backups and production configuration.

Final architecture

```
Backend: Node.js + Express
Database: MySQL 8+ with InnoDB
Authentication: secure cookie sessions + MySQL session store
Passwords: bcrypt
Authorization: owner, parish, centre and group scopes
Data: shared normalized tables with parish_id
Financial behavior: records already-received contributions only
Integrity: audit logs and contribution voiding
API: resource endpoints, dashboards and reports
```

This architecture is simple, maintainable, testable and aligned with the Finance ERP frontend while preserving tenant isolation, financial accountability and future extensibility.